

Software engineering

Lecture 2

Software engineering

- ▶ Software engineering is an engineering discipline that is concerned with all aspects of software production from the early stages of system specification through to maintaining the system after it has gone into use.
- ▶ Engineering discipline
 - ▶ Using appropriate theories and methods to solve problems bearing in mind organizational and financial constraints.
- ▶ All aspects of software production
 - ▶ Not just technical process of development. Also project management and the development of tools, methods etc. to support software production.

Software engineering

- ▶ The economies of ALL developed nations are dependent on software.
- ▶ More and more systems are software controlled
- ▶ Software engineering is concerned with theories, methods and tools for professional software development.
- ▶ Expenditure on software represents a significant fraction of GNP in all developed countries.

Importance of software engineering

- ▶ More and more, individuals and society rely on advanced software systems. We need to be able to produce reliable and trustworthy systems economically and quickly.
- ▶ It is usually cheaper, in the long run, to use software engineering methods and techniques for software systems rather than just write the programs as if it was a personal programming project. For most types of system, the majority of costs are the costs of changing the software after it has gone into use.

Software costs

- ▶ Software costs often dominate computer system costs. The costs of software on a PC are often greater than the hardware cost.
- ▶ Software costs more to maintain than it does to develop. For systems with a long life, maintenance costs may be several times development costs.
- ▶ Software engineering is concerned with cost-effective software development.

Software project failure

► **Increasing system complexity**

- As new software engineering techniques help us to build larger, more complex systems, the demands change. Systems have to be built and delivered more quickly; larger, even more complex systems are required; systems have to have new capabilities that were previously thought to be impossible.

► **Failure to use software engineering methods**

- It is fairly easy to write computer programs without using software engineering methods and techniques. Many companies have drifted into software development as their products and services have evolved. They do not use software engineering methods in their everyday work. Consequently, their software is often more expensive and less reliable than it should be.

Software products

▶ Generic products

- ▶ Stand-alone systems that are marketed and sold to any customer who wishes to buy them.
- ▶ Examples - PC software such as graphics programs, project management tools; CAD software; software for specific markets such as appointments systems for dentists.

▶ Customized products

- ▶ Software that is commissioned by a specific customer to meet their own needs.
- ▶ Examples - embedded control systems, air traffic control software, traffic monitoring systems.

Product specification

- ▶ **Generic products**

- ▶ The specification of what the software should do is owned by the software developer and decisions on software change are made by the developer.

- ▶ **Customized products**

- ▶ The specification of what the software should do is owned by the customer for the software and they make decisions on software changes that are required.

Essential attributes of good software

Product characteristic	Description
Maintainability	Software should be written in such a way so that it can evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.
Dependability and security	Software dependability includes a range of characteristics including reliability, security and safety. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.
Efficiency	Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilisation, etc.
Acceptability	Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable and compatible with other systems that they use.

Software process activities

- ▶ Software **specification**, where customers and engineers define the software that is to be produced and the constraints on its operation.
- ▶ Software **development**, where the software is designed and programmed.
- ▶ Software **validation**, where the software is checked to ensure that it is what the customer requires.
- ▶ Software **evolution**, where the software is modified to reflect changing customer and market requirements.

General issues that affect software

- ▶ Heterogeneity
 - ▶ Increasingly, systems are required to operate as distributed systems across networks that include different types of computer and mobile devices.
- ▶ Business and social change
 - ▶ Business and society are changing incredibly quickly as emerging economies develop and new technologies become available. They need to be able to change their existing software and to rapidly develop new software.

General issues that affect software

- ▶ Security and trust
 - ▶ As software is intertwined with all aspects of our lives, it is essential that we can trust that software.
- ▶ Scale
 - ▶ Software has to be developed across a very wide range of scales, from very small embedded systems in portable or wearable devices through to Internet-scale, cloud-based systems that serve a global community.

Software engineering diversity

- ▶ There are many different types of software system and there is no universal set of software techniques that is applicable to all of these.
- ▶ The software engineering methods and tools used depend on the type of application being developed, the requirements of the customer and the background of the development team.

Application types

- ▶ Stand-alone applications
 - ▶ These are application systems that run on a local computer, such as a PC. They include all necessary functionality and do not need to be connected to a network.
- ▶ Interactive transaction-based applications
 - ▶ Applications that execute on a remote computer and are accessed by users from their own PCs or terminals. These include web applications such as e-commerce applications.
- ▶ Embedded control systems
 - ▶ These are software control systems that control and manage hardware devices. Numerically, there are probably more embedded systems than any other type of system.

Application types

- ▶ Batch processing systems
 - ▶ These are business systems that are designed to process data in large batches. They process large numbers of individual inputs to create corresponding outputs.
- ▶ Entertainment systems
 - ▶ These are systems that are primarily for personal use and which are intended to entertain the user.
- ▶ Systems for modelling and simulation
 - ▶ These are systems that are developed by scientists and engineers to model physical processes or situations, which include many, separate, interacting objects.

Application types

- ▶ Data collection systems
 - ▶ These are systems that collect data from their environment using a set of sensors and send that data to other systems for processing.
- ▶ Systems of systems
 - ▶ These are systems that are composed of a number of other software systems.

Software engineering fundamentals

- ▶ Some fundamental principles apply to all types of software system, irrespective of the development techniques used:
 - ▶ Systems should be developed using a managed and understood development process. Of course, different processes are used for different types of software.
 - ▶ Dependability and performance are important for all types of system.
 - ▶ Understanding and managing the software specification and requirements (what the software should do) are important.
 - ▶ Where appropriate, you should reuse software that has already been developed rather than write new software.

Internet software engineering

- ▶ The Web is now a platform for running application and organizations are increasingly developing web-based systems rather than local systems.
- ▶ Web services allow application functionality to be accessed over the web.
- ▶ Cloud computing is an approach to the provision of computer services where applications run remotely on the 'cloud'.
 - ▶ Users do not buy software buy pay according to use.

Web-based software engineering

- ▶ Web-based systems are complex distributed systems but the fundamental principles of software engineering discussed previously are as applicable to them as they are to any other types of system.
- ▶ The fundamental ideas of software engineering apply to web-based software in the same way that they apply to other types of software system.

Web software engineering

- ▶ Software reuse
 - ▶ Software reuse is the dominant approach for constructing web-based systems. When building these systems, you think about how you can assemble them from pre-existing software components and systems.
- ▶ Incremental and agile development
 - ▶ Web-based systems should be developed and delivered incrementally. It is now generally recognized that it is impractical to specify all the requirements for such systems in advance.

Web software engineering

- ▶ Service-oriented systems
 - ▶ Software may be implemented using service-oriented software engineering, where the software components are stand-alone web services.
- ▶ Rich interfaces
 - ▶ Interface development technologies such as AJAX and HTML5 have emerged that support the creation of rich interfaces within a web browser.

Dual Role of Software:

- ▶ **1. As a product -**
 - ▶ It delivers the computing potential across network of Hardware.
 - ▶ It enables the Hardware to deliver the expected functionality.
 - ▶ It acts as information transformer because it produces, manages, acquires, modifies, displays, or transmits information.
- ▶ **2. As a vehicle for delivering a product -**
 - ▶ It provides system functionality (e.g., payroll system)
 - ▶ It controls other software (e.g., an operating system)
 - ▶ It helps build other software (e.g., software tools)

Objectives of Software Engineering:

- ▶ **Maintainability -**
It should be feasible for the software to evolve to meet changing requirements.
- ▶ **Correctness -**
A software product is correct, if the different requirements as specified in the SRS document have been correctly implemented.
- ▶ **Reusability -**
A software product has good reusability, if the different modules of the product can easily be reused to develop new products.
- ▶ **Testability -**
Here software facilitates both the establishment of test criteria and the evaluation of the software with respect to those criteria.

Objectives of Software Engineering:

- ▶ **Reliability -**

It is an attribute of software quality. The extent to which a program can be expected to perform its desired function, over an arbitrary time period.

- ▶ **Portability -**

In this case, software can be transferred from one computer system or environment to another.

- ▶ **Adaptability -**

In this case, software allows differing system constraints and user needs to be satisfied by making changes to the software.

Program vs Software Product:

- ▶ Program is a **set of instruction** related each other where as Software Product is **a collection of program** designed for specific task.
- ▶ Programs are usually **small in size** where as Software Products are usually **large in size**.
- ▶ Programs are developed by **individuals** that means single user where as Software Product are developed by **large no of users**.
- ▶ In program, there is **no documentation** or lack in proper documentation.
In Software Product, **Proper documentation** and well documented and user manual prepared.

Program vs Software Product:

- ▶ Development of program is **Unplanned, not Systematic** etc but Development of Software Product is **well Systematic**, organized, planned approach.
- ▶ Programs provide **Limited functionality and less features** where as Software Products provides **more functionality as they are big in size (lines of codes) more options and features.**

Software Evolution

- ▶ The process of developing a software product using software engineering principles and methods is referred to as **software evolution**.

This includes the initial development of software and its maintenance and updates, till desired software product is developed, which satisfies the expected requirements.

- ▶ Evolution starts from the **requirement gathering process**. After which developers create a prototype of the intended software and show it to the users to get their feedback at the early stage of software product development. The users suggest changes, on which several consecutive updates and maintenance keep on changing too. This process changes to the original software, till the desired software is accomplished.
- ▶ Even after the user has desired software in hand, the advancing technology and the changing requirements force the software product to change accordingly. Re-creating software from scratch and to go one-on-one with requirement is not feasible. The only feasible and economical solution is to update the existing software so that it matches the latest requirements.



Software Evolution Laws

- ▶ Lehman has given laws for software evolution. He divided the software into three different categories:
- ▶ **S-type (static-type)** - This is a software, which works strictly according to defined specifications and solutions. The solution and the method to achieve it, both are immediately understood before coding. The s-type software is least subjected to changes hence this is the simplest of all. For example, calculator program for mathematical computation.

- ▶ **P-type (practical-type)** - This is a software with a collection of procedures. This is defined by exactly what procedures can do. In this software, the specifications can be described but the solution is not obvious instantly. For example, gaming software.
- ▶ **E-type (embedded-type)** - This software works closely as the requirement of real-world environment. This software has a high degree of evolution as there are various changes in laws, taxes etc. in the real world situations. For example, Online trading software.

E-Type software evolution

- ▶ Lehman has given eight laws for E-Type software evolution -
 - ▶ **Continuing change** - An E-type software system must continue to adapt to the real world changes, else it becomes progressively less useful.
 - ▶ **Increasing complexity** - As an E-type software system evolves, its complexity tends to increase unless work is done to maintain or reduce it.
 - ▶ **Conservation of familiarity** - The familiarity with the software or the knowledge about how it was developed, why was it developed in that particular manner etc. must be retained at any cost, to implement the changes in the system.

- ▶ **Continuing growth-** In order for an E-type system intended to resolve some business problem, its size of implementing the changes grows according to the lifestyle changes of the business.
- ▶ **Reducing quality** - An E-type software system declines in quality unless rigorously maintained and adapted to a changing operational environment.
- ▶ **Feedback systems-** The E-type software systems constitute multi-loop, multi-level feedback systems and must be treated as such to be successfully modified or improved.
- ▶ **Self-regulation** - E-type system evolution processes are self-regulating with the distribution of product and process measures close to normal.
- ▶ **Organizational stability** - The average effective global activity rate in an evolving E- type system is invariant over the lifetime of the product.

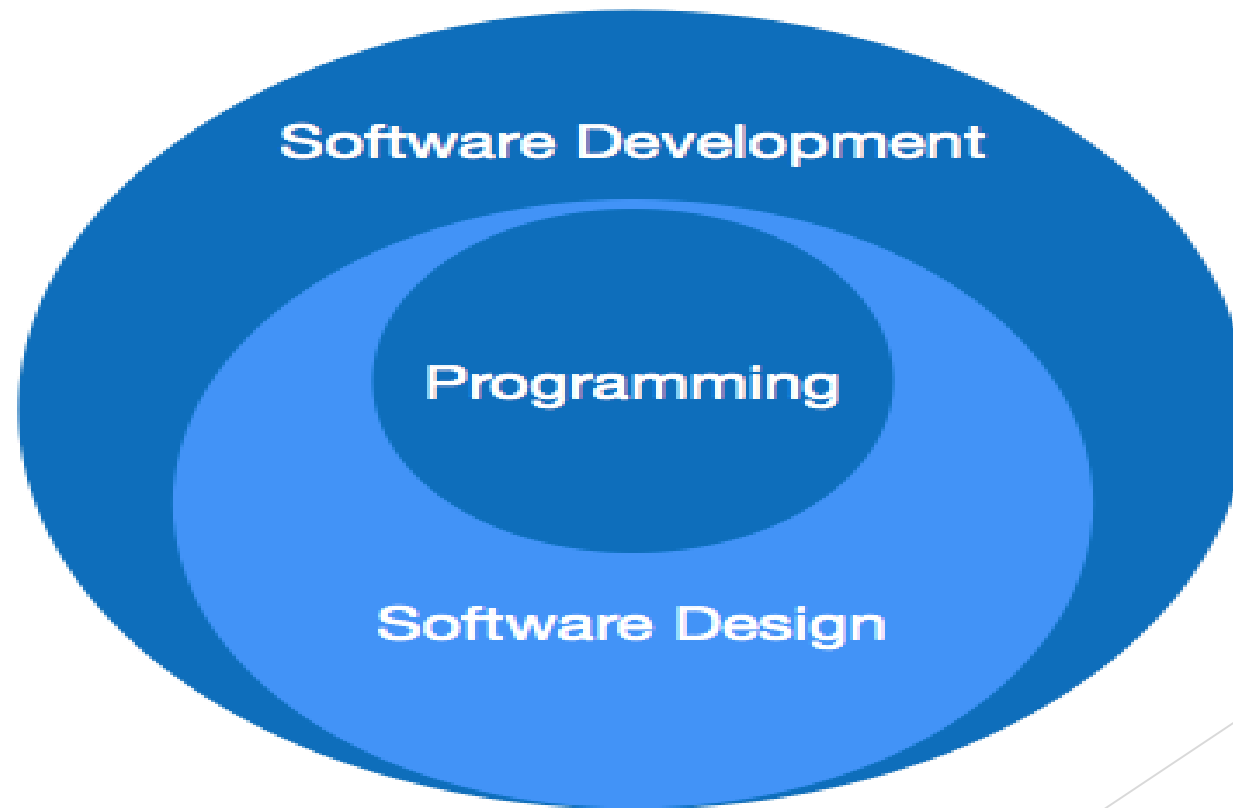
Software Paradigms

- Software paradigms refer to the methods and steps, which are taken while designing the software.

There are many methods proposed and are in work today, but we need to see where in the software engineering these paradigms stand.

These can be combined into various categories, though each of them is contained in one another

- Programming paradigm is a subset of Software design paradigm which is further a subset of Software development paradigm.



Software Development Paradigm

- ▶ This Paradigm is known as software engineering paradigms where all the engineering concepts pertaining to the development of software are applied. It includes various researches and requirement gathering which helps the software product to build. It consists of -
 - ▶ Requirement gathering
 - ▶ Software design
 - ▶ Programming

► **Software Design Paradigm**

This paradigm is a part of Software Development and includes

-

- Design
- Maintenance
- Programming

► **Programming Paradigm**

This paradigm is related closely to programming aspect of software development. This includes -

- Coding
- Testing
- Integration

Characteristics of good software

- ▶ A software product can be judged by what it offers and how well it can be used. This software must satisfy on the following grounds:
- ▶ Operational
- ▶ Transitional
- ▶ Maintenance
- ▶ Well-engineered and crafted software is expected to have the following characteristics

Operational

This tells us how well software works in operations. It can be measured on:

- ▶ Budget
- ▶ Usability
- ▶ Efficiency
- ▶ Correctness
- ▶ Functionality
- ▶ Dependability
- ▶ Security
- ▶ Safety

Maintenance

This aspect briefs about how well a software has the capabilities to maintain itself in the ever-changing environment:

- ▶ Modularity
 - ▶ Maintainability
 - ▶ Flexibility
 - ▶ Scalability
-
- ▶ In short, Software engineering is a branch of computer science, which uses well-defined engineering concepts required to produce efficient, durable, scalable, in-budget and on-time software products.

Transitional

This aspect is important when the software is moved from one platform to another:

- ▶ Portability
- ▶ Interoperability
- ▶ Reusability
- ▶ Adaptability

Management

- Effectiveness and Efficiency
 - Effectiveness: The achievement of objectives
 - Efficiency: The achievement of ends with least amount of resources
- Managing is concerned with productivity
- “Productivity” ?
 - $\text{Productivity} = \text{outputs/inputs}$
 - Productivity can be increased by
 - Using “same” amount of resource but more output
 - Using less resources but giving “same” output
 - Using less resources but more output
- Business vs non-business

Management Approaches

- Scientific Approach - Taylor
 - To increase efficiency (in production) by applying scientific methods
- Sociological Approach – Emile Durkheim
 - Groups, by establishing their values and norms control human conduct in any social organization
- Socio-Technical Approach – Elton Mayo
 - Business operations are a matter not merely of machinery and methods but also of gearing these with the social system to develop a complete socio-technical system

Five Functions of Management

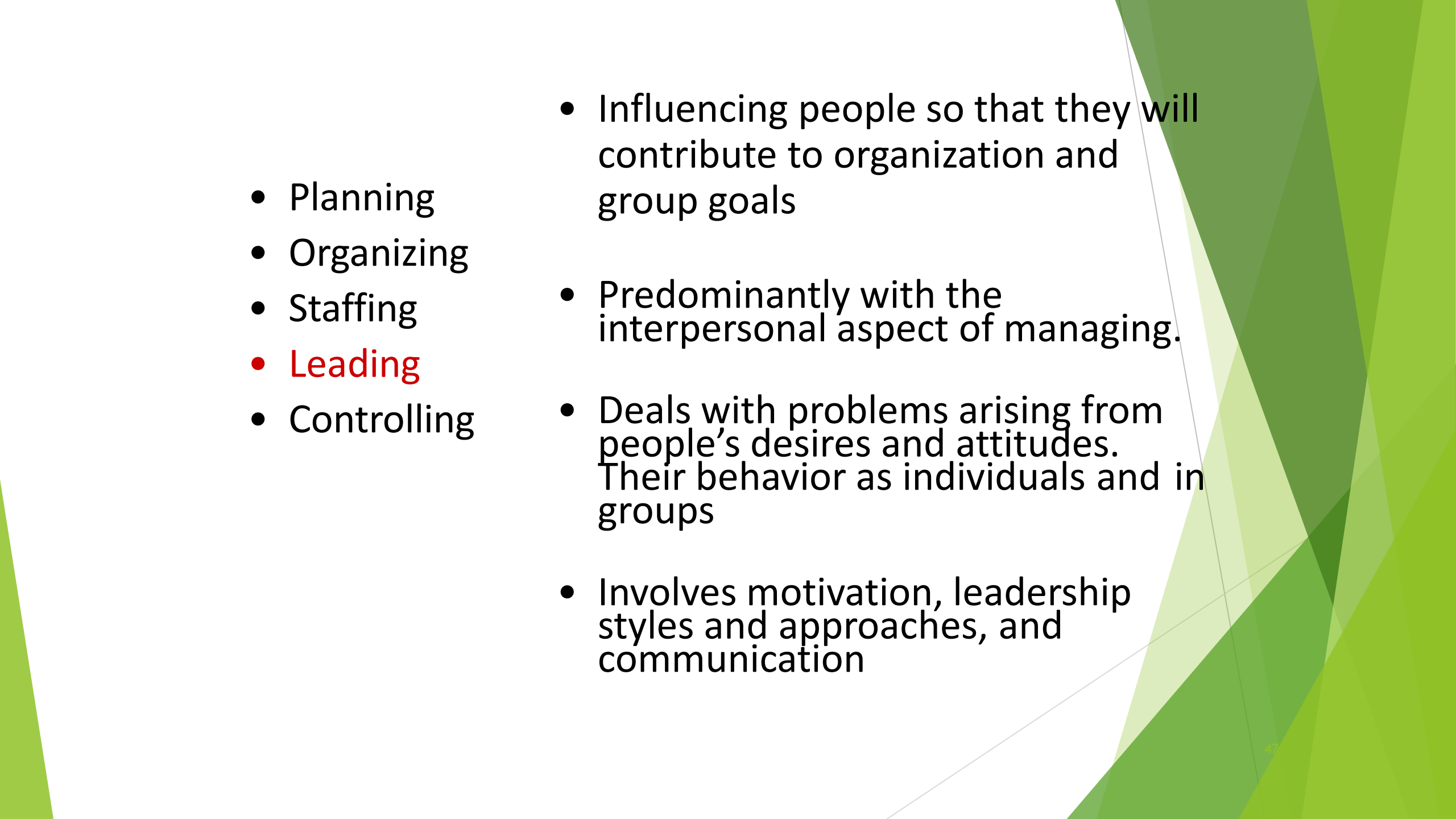
- **Planning**
 - Organizing
 - Staffing
 - Leading
 - Controlling
- Involves selecting missions and objectives and the actions to achieve them
 - Requires decisions making that is, choosing future courses of action from among alternatives

- Planning
- **Organizing**
- Staffing
- Leading
- Controlling

- People working together in groups to achieve some goal must have roles to play
- Organizing involves establishing an “intentional” structure of roles for people to fill in an organization.

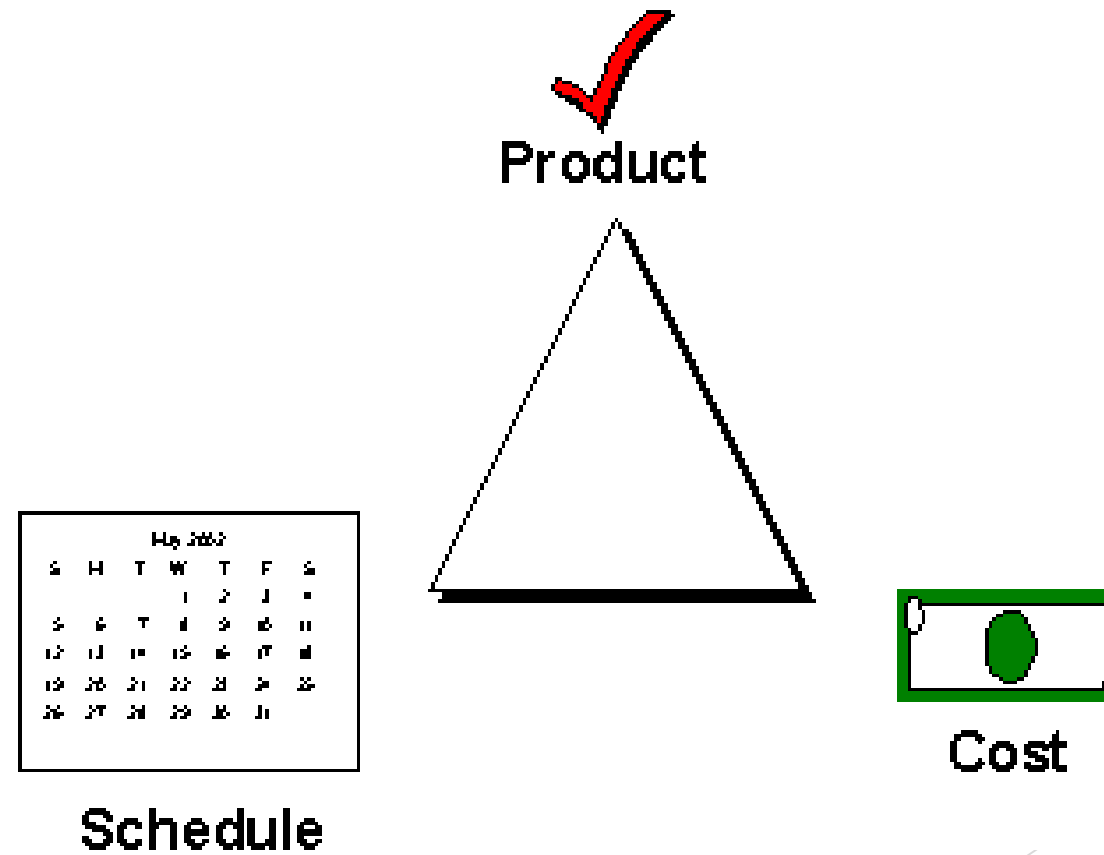
– intentional in the sense of making sure that all the tasks necessary to accomplish goals are assigned.

- Planning
 - Organizing
 - **Staffing**
 - Leading
 - Controlling
- Involves filling, and keeping filled the positions in the organization structure.
 - This is done by identifying work-force requirements, inventorying the people available; and
 - Recruiting, selecting, placing, promoting, appraising, planning the careers of, compensating, and training or otherwise developing both candidates and current jobholders so that tasks are accomplished effectively and efficiently

- 
- The background of the slide features abstract, overlapping green geometric shapes in various shades, creating a modern and dynamic visual effect.
- Planning
 - Organizing
 - Staffing
 - **Leading**
 - Controlling
- Influencing people so that they will contribute to organization and group goals
 - Predominantly with the interpersonal aspect of managing.
 - Deals with problems arising from people's desires and attitudes. Their behavior as individuals and in groups
 - Involves motivation, leadership styles and approaches, and communication

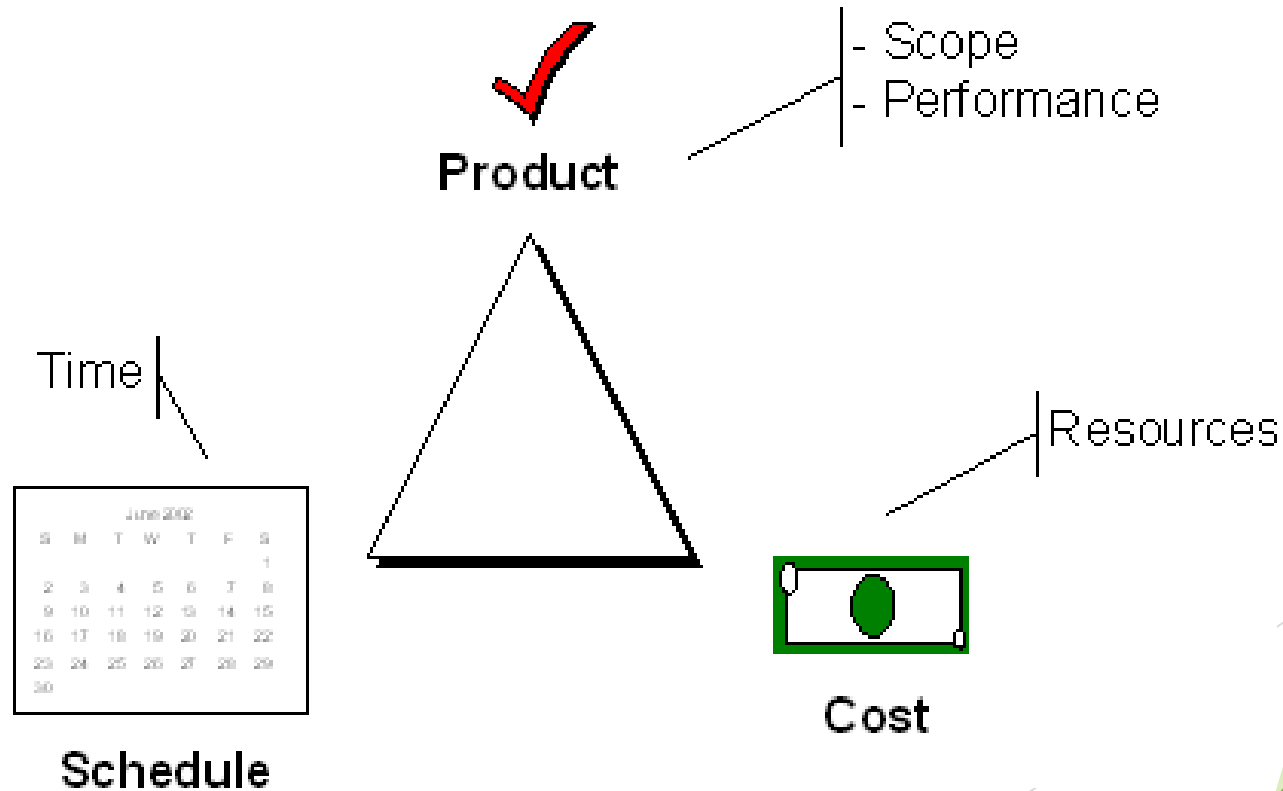
- Planning
 - Organizing
 - Staffing
 - Leading
 - **Controlling**
- Measuring and correcting individual and organizational performance to ensure that events conform to plans
 - Involves measuring performance against goals and plans, showing where deviations from standard exist, and helping to correct them

Project Management Trade-off Triangle

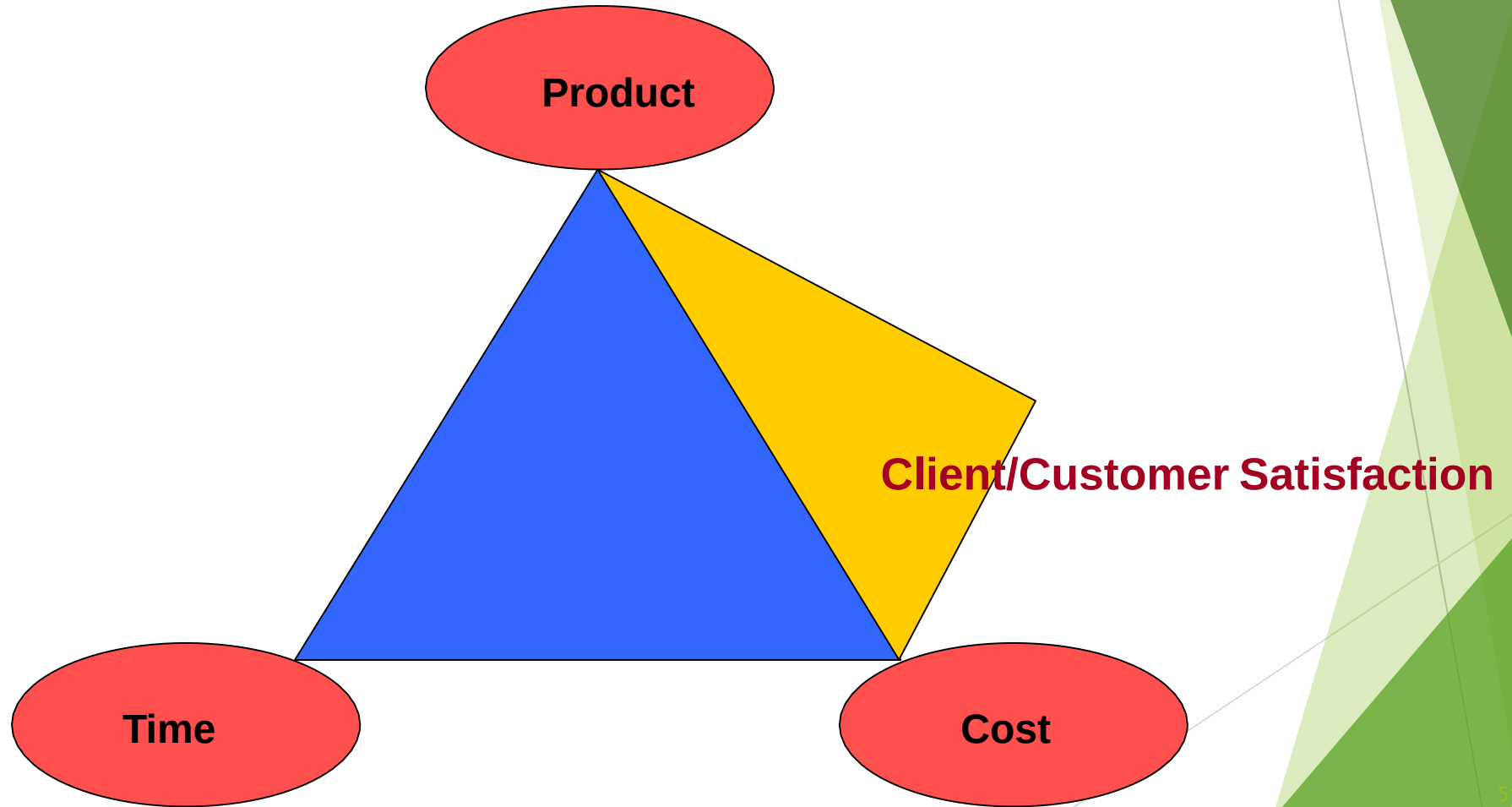


Project Management Trade-off Triangle

- Know which of these are fixed & variable for every project



Project Success



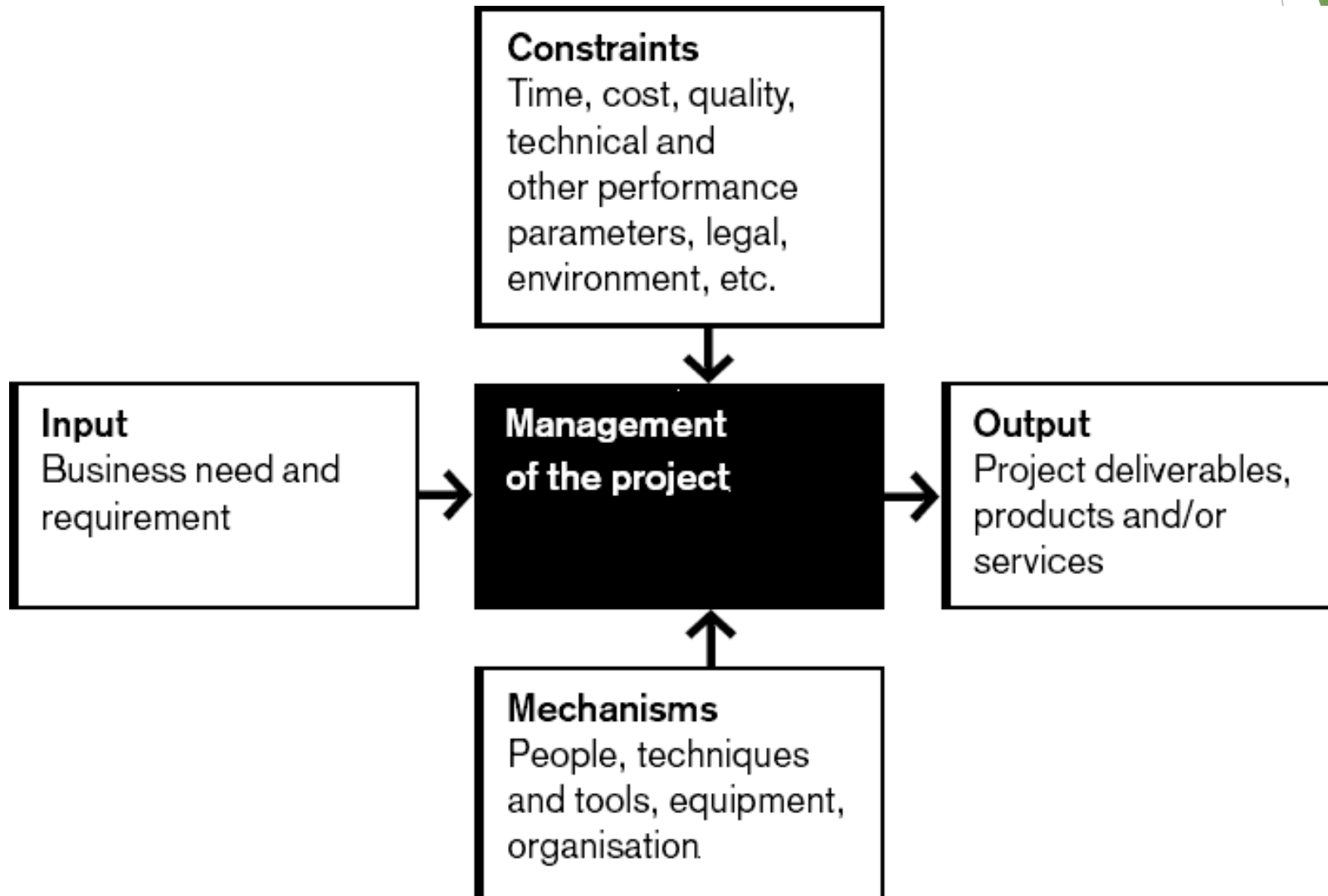
Project management

- The application of knowledge skills, tools, and techniques to project activities to meet project requirements. (PMI)
 - to meet or exceed stakeholder needs and expectations from a project!!!
- Managing a project includes:
 - Identifying requirements
 - Establishing clear and achievable objectives
 - Balancing the competing demands for quality, scope, time and cost
 - Adapting the specifications, plans, and approach to the different concerns and
 - expectations of the various stakeholders.

Programme Management

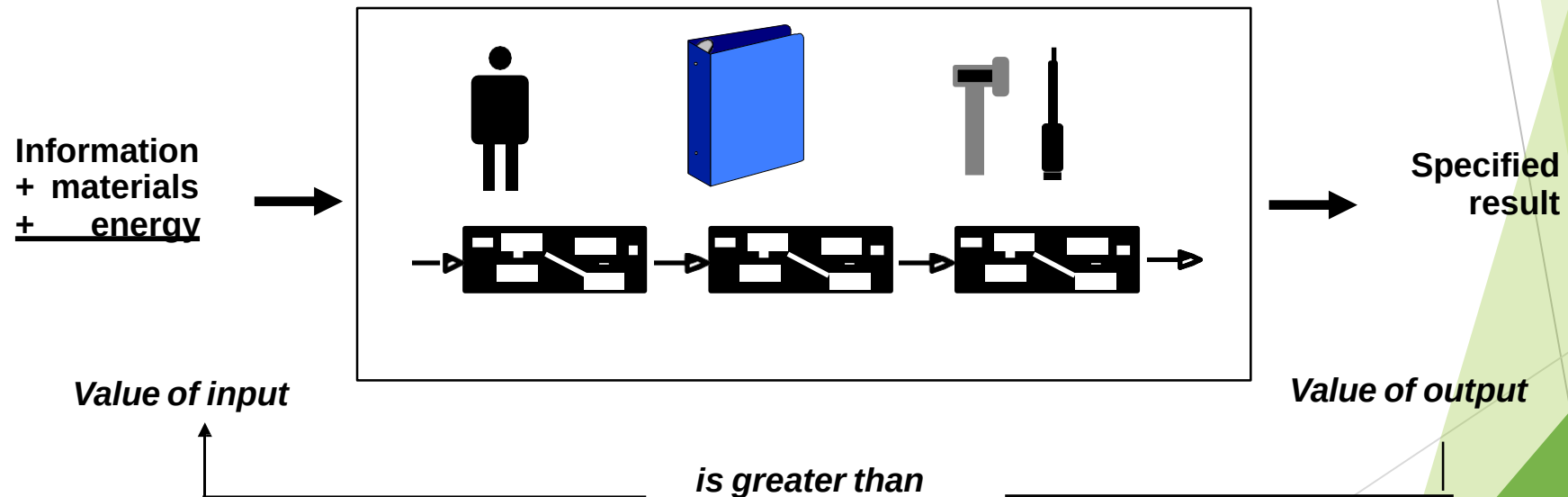
- A programme is a collection of projects related to some extent to a common objective.
- Programme Management is the effective management of that programme.
- It can variously cover managing any or all of the following:
 - a portfolio of projects related to some common objective;
 - an organization's business strategy which is to be implemented through projects;
 - the interdependencies between a number of projects;
 - resource allocation amongst a portfolio of projects.
- Portfolio Management
 - the management of a number of projects that do not share a common objective.

Project Management Process



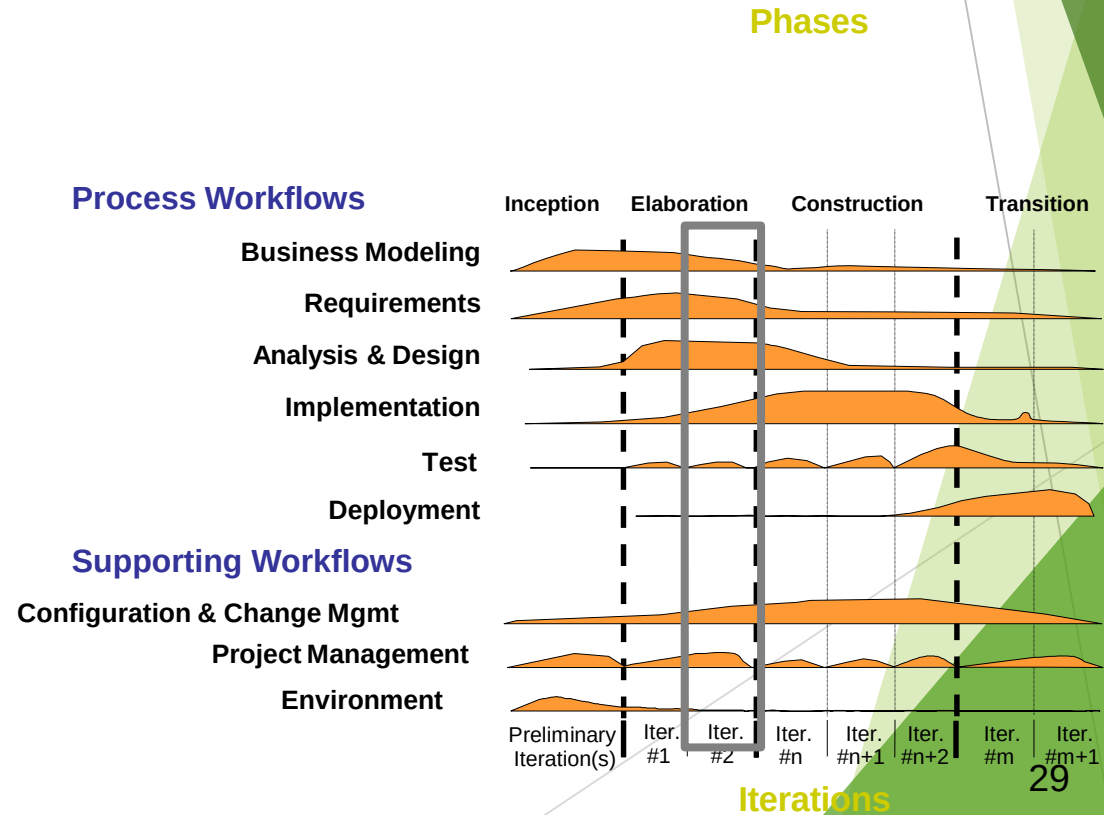
What is a Process?

- process : a bounded set of interrelated activities that takes one or more kinds of inputs and creates outputs that are of value to the customer by means of one or more transformations



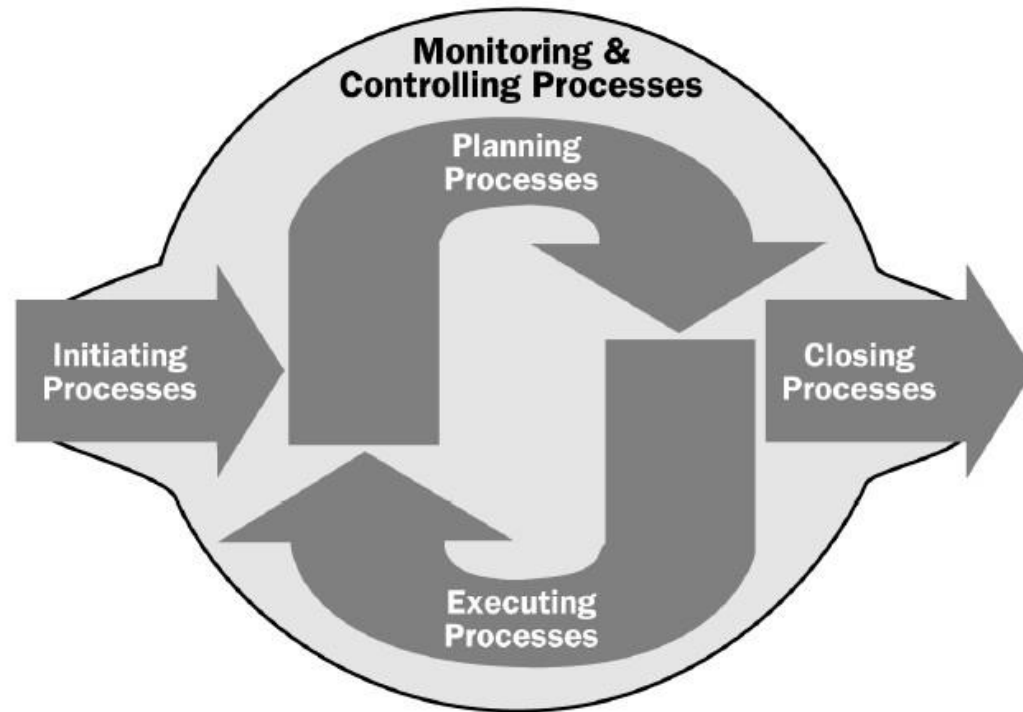
Type of Processes

- PM processes
 - describing and organizing the work of the project
- Product-oriented processes
 - specifying and building the project's product



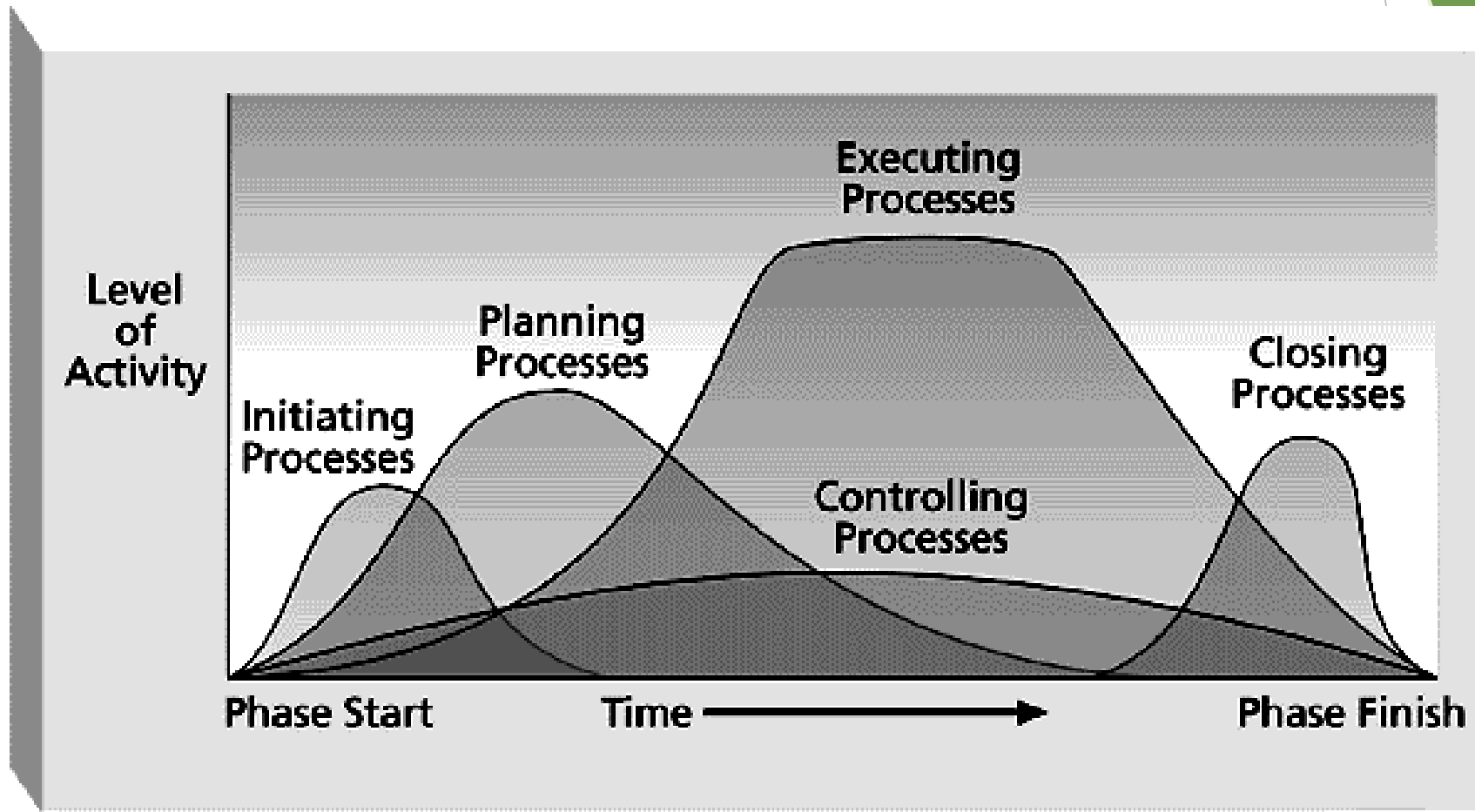
Project Management processes

- Project management is accomplished through the application and management of the following processes:
 - Initiating
 - Planning
 - Executing
 - Monitoring and Controlling
 - Closing



- These can be repeated for every phase of a project

Process Groups



Planning Process

Devising and maintaining a workable scheme to accomplish the business need that the project was undertaken to address

- Scope Planning
- Scope Definition
- Activity Definition
- Activity Sequencing
- Activity Duration Estimating
- Resource Planning
- Cost Estimating
- Cost Budgeting
- Risk Planning
- Schedule Development
- Quality Planning
- Communications Planning
- Organization Planning
- Staff Acquisition
- Procurement Planning
- Project Plan Development

Executing Process

Coordinating people and other resources to carry out the plan

- Project Plan Execution
- Scope Verification
- Quality Assurance
- Team Development
- Information Distribution
- Solicitation
- Source Selection
- Contract Administration

Controlling Process

Ensuring that project objectives are met by monitoring and measuring progress and taking corrective measures when necessary

- Overall Change Control
- Scope Change Control
- Schedule Control
- Cost Control
- Quality Control
- Performance Reporting
- Risk Response Control

Software engineering ethics

Software engineering ethics

- ▶ Software engineering involves wider responsibilities than simply the application of technical skills.
- ▶ Software engineers must behave in an honest and ethically responsible way if they are to be respected as professionals.
- ▶ Ethical behaviour is more than simply upholding the law but involves following a set of principles that are morally correct.

Issues of professional responsibility

- ▶ Confidentiality
 - ▶ Engineers should normally respect the confidentiality of their employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.
- ▶ Competence
 - ▶ Engineers should not misrepresent their level of competence. They should not knowingly accept work which is outwith their competence.

Issues of professional responsibility

- ▶ Intellectual property rights
 - ▶ Engineers should be aware of local laws governing the use of intellectual property such as patents, copyright, etc. They should be careful to ensure that the intellectual property of employers and clients is protected.
- ▶ Computer misuse
 - ▶ Software engineers should not use their technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine, say) to extremely serious (dissemination of viruses).

ACM/IEEE Code of Ethics

- ▶ The professional societies in the US have cooperated to produce a code of ethical practice.
- ▶ Members of these organisations sign up to the code of practice when they join.
- ▶ The Code contains eight Principles related to the behaviour of and decisions made by professional software engineers, including practitioners, educators, managers, supervisors and policy makers, as well as trainees and students of the profession.

Rationale for the code of ethics

- ▶ *Computers have a central and growing role in commerce, industry, government, medicine, education, entertainment and society at large. Software engineers are those who contribute by direct participation or by teaching, to the analysis, specification, design, development, certification, maintenance and testing of software systems.*
- ▶ *Because of their roles in developing software systems, software engineers have significant opportunities to do good or cause harm, to enable others to do good or cause harm, or to influence others to do good or cause harm. To ensure, as much as possible, that their efforts will be used for good, software engineers must commit themselves to making software engineering a beneficial and respected profession.*

The ACM/IEEE Code of Ethics

Software Engineering Code of Ethics and Professional Practice

ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices

PREAMBLE

The short version of the code summarizes aspirations at a high level of the abstraction; the clauses that are included in the full version give examples and details of how these aspirations change the way we act as software engineering professionals. Without the aspirations, the details can become legalistic and tedious; without the details, the aspirations can become high sounding but empty; together, the aspirations and the details form a cohesive code.

Software engineers shall commit themselves to making the analysis, specification, design, development, testing and maintenance of software a beneficial and respected profession. In accordance with their commitment to the health, safety and welfare of the public, software engineers shall adhere to the following Eight Principles:

Ethical principles

1. PUBLIC - Software engineers shall act consistently with the public interest.
2. CLIENT AND EMPLOYER - Software engineers shall act in a manner that is in the best interests of their client and employer consistent with the public interest.
3. PRODUCT - Software engineers shall ensure that their products and related modifications meet the highest professional standards possible.
4. JUDGMENT - Software engineers shall maintain integrity and independence in their professional judgment.
5. MANAGEMENT - Software engineering managers and leaders shall subscribe to and promote an ethical approach to the management of software development and maintenance.
6. PROFESSION - Software engineers shall advance the integrity and reputation of the profession consistent with the public interest.
7. COLLEAGUES - Software engineers shall be fair to and supportive of their colleagues.
8. SELF - Software engineers shall participate in lifelong learning regarding the practice of their profession and shall promote an ethical approach to the practice of the profession.